

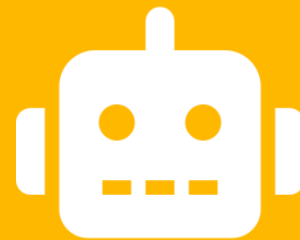
UNDERGRADUATE COURSE

Building Blocks of a GenAI Transformer

CLASS 2

Pre-Training, SFT, RLHF, RLVR & Multi-Modality

Instructor: Naresh Shah



Today's Agenda

Class 1 Recap

- Tokenisation & Embedding
- Model Inference
- Dialogue State Inference
- Response Generation
- Intent & Intent Classification
- Memory

Class 2 Topics

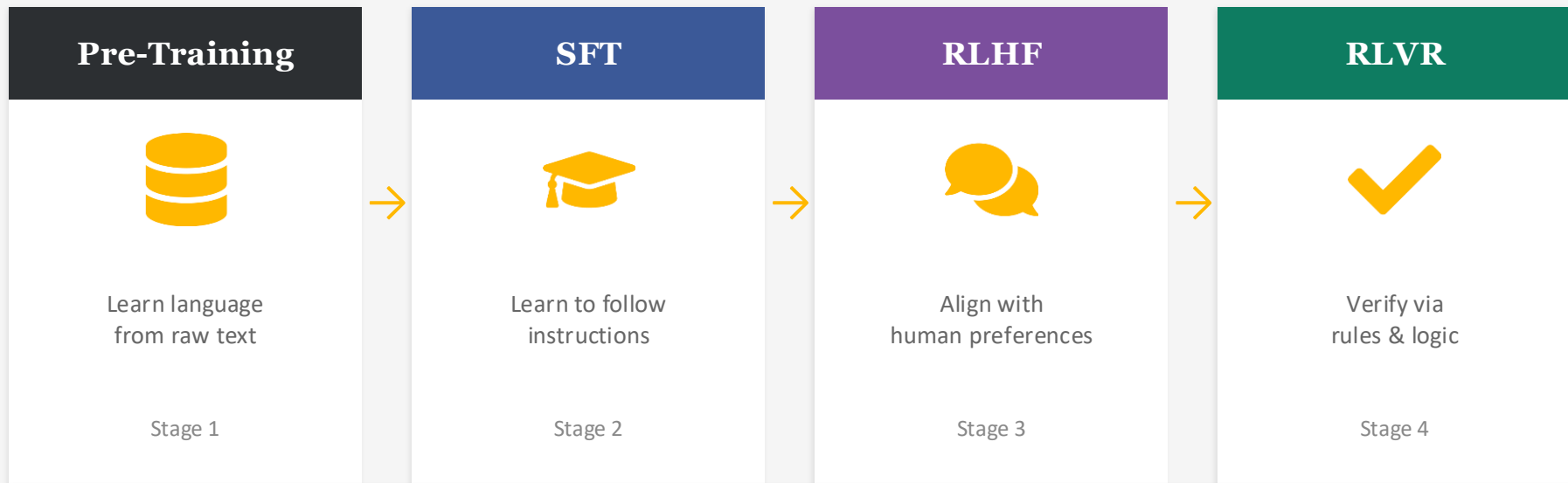
- 1 Pre-Training vs SFT vs RLHF vs RLVR
- 2 Multi-Modality in LLMs
- 3 RLHF Example: Function Calling
- 4 RLVR Example: Code Generation
- 5 Capabilities Gained

SECTION 1

Training Paradigms

Pre-Training, SFT, RLHF & RLVR

The LLM Training Pipeline



Student Analogy: The Learning Journey

Pre-Training = Reading every book in a library → SFT = Getting a tutor → RLHF = Learning from a mentor's preferences → RLVR = Grading with an answer key

Pre-Training: Next-Token Prediction



How Next-Token Prediction Works

- 1 Input: "The quick brown fox"
- 2 Model predicts next token distribution
- 3 Ground truth: "jumps" → compute loss
- 4 Backpropagate → update weights
- 5 Repeat trillions of times

15T+

tokens in Llama 3's training data
(Common Crawl, Books, Code, Wikipedia)

Causal Attention Mask

Each token can only attend to tokens BEFORE it (left-to-right). Future tokens are masked out (set to $-\infty$). This forces the model to learn genuinely predictive patterns.



Analogy: Learning Language Like a Child

A child hears "Hello, how are yo—" and predicts "you!" Pre-training works the same way: learn patterns through billions of predictions.

Supervised Fine-Tuning (SFT)



How SFT Works

- 1 Curate (instruction, response) pairs from human annotators
- 2 Fine-tune pre-trained model on these pairs
- 3 Loss applied ONLY on response tokens (not instruction)
- 4 Model learns instruction-following format

Example SFT Pair

Instruction:

"Explain quantum entanglement in simple terms."

Response:

"Quantum entanglement is when two particles become connected so that measuring one instantly affects the other, no matter how far apart..."

Key Papers

InstructGPT

13K pairs → 1.3B model beats 175B GPT-3 (Ouyang et al., 2022)

FLAN

60+ tasks → zero-shot generalization (Wei et al., 2022)

Alpaca

52K synthetic pairs, <\$600 → near-GPT-3.5 (Stanford, 2023)



Analogy: The Tutor

Pre-training = reading every textbook. SFT = a tutor gives practice problems with ideal solutions. The student learns HOW to apply knowledge.

RLHF: The 3-Stage Pipeline

1

Generate & Rank

Model produces multiple responses.
Human annotators rank by preference.
"Which response is better?"



2

Train Reward Model

Train a separate model to predict
human preferences from rankings.
Input: (prompt, response) → Score 0-1



3

PPO Optimization

Use reward model to optimize LLM.
Maximize reward while staying close
to original SFT model (KL penalty).



Key Concepts

PPO Clipping:

Limits how much the model can change in one step
— prevents instability

KL Penalty:

Keeps the new model close to the SFT model —
prevents mode collapse

3H Framework:

Helpful, Harmless, Honest (Anthropic)

Constitutional AI

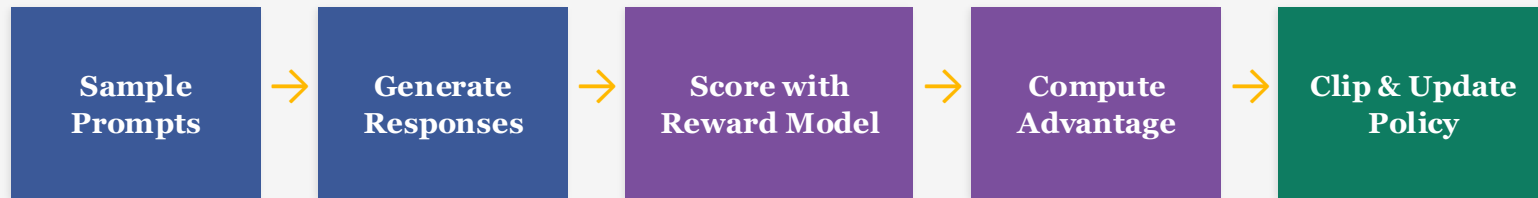
Anthropic's alternative: replace human annotators with
AI feedback guided by a set of principles ('constitution').
Dramatically reduces cost while improving both
helpfulness AND safety.



Analogy: Training a Dog with Treats

Reward Model = deciding which behaviors you like. PPO = giving treats for good behavior, but only if close to what the dog already knows.

How PPO Works: A Visual Walkthrough



The Clipping Mechanism

PPO computes: $\text{ratio} = P(\text{new}) / P(\text{old})$

If $\text{ratio} > 1.2$ or < 0.8 , it gets clipped.

This prevents the model from changing too much in a single update — like a speed limit for learning.

Without clipping → reward hacking, mode collapse

The KL Divergence Penalty

$\text{reward} = \text{RM_score} - \beta \times \text{KL}(\text{new} || \text{old})$

KL divergence measures how different the new model is from the original SFT model.

High KL → big penalty → model stays grounded

Prevents: "forget everything and just maximize reward"



Analogy: Learning to Drive

Clipping = speed limit (can't turn the wheel 90°). KL penalty = GPS keeping you on-route (explore side streets but not a different city). Together: safe, gradual improvement.

RLVR: Verifiable Rewards Replace Human Judges



The RLHF Problem RLVR Solves

RLHF's reward model can be gamed:

- Model learns 'long answers get higher scores'
- Generates verbose, repetitive text (reward hacking)

RLVR's solution:

- Use automatic, objective verification
- Math: does answer = ground truth?
- Code: do unit tests pass?
- **Impossible to fake correctness**

GRPO Algorithm (DeepSeek)

- 1 Generate K responses per prompt (e.g., K=4)
- 2 Verify all responses automatically
- 3 Correct → reward +1, Wrong → reward 0
- 4 Normalize: advantage = $(r - \text{mean}) / \text{std}$
- 5 Update policy (no critic/reward model needed!)



Emergent Chain-of-Thought

DeepSeek-R1-Zero (trained with RLVR only, no SFT) spontaneously developed step-by-step reasoning, self-verification, and backtracking — because thinking through problems increases the reward signal. No one taught it to reason; it discovered reasoning is useful.



Analogy: Grading with an Answer Key

RLHF = hiring a tutor to grade (can be gamed). RLVR = using an answer key (objective, can't be faked). Students genuinely learn.

Side-by-Side: Pre-Training vs SFT vs RLHF vs RLVR

Dimension	Pre-Training	SFT	RLHF	RLVR
Objective	Next-token prediction	Instruction following	Human preference	Verifiable correctness
Data	Web crawls (trillions)	Curated pairs (10K-100K)	Human rankings (50K)	Auto-generated test cases
Human Labor	None	High	High	None
Cost	\$\$\$\$\$ (compute)	\$\$\$ (annotation)	\$\$\$\$ (annotation)	\$\$ (compute only)
Can Be Gamed?	N/A	N/A	Yes (reward hacking)	No (ground truth)
Key Benefit	Broad knowledge	Task alignment	Value alignment	Reasoning accuracy
Analogy	 Reading the library	 Getting a tutor	 Training with treats	 Grading with answer key

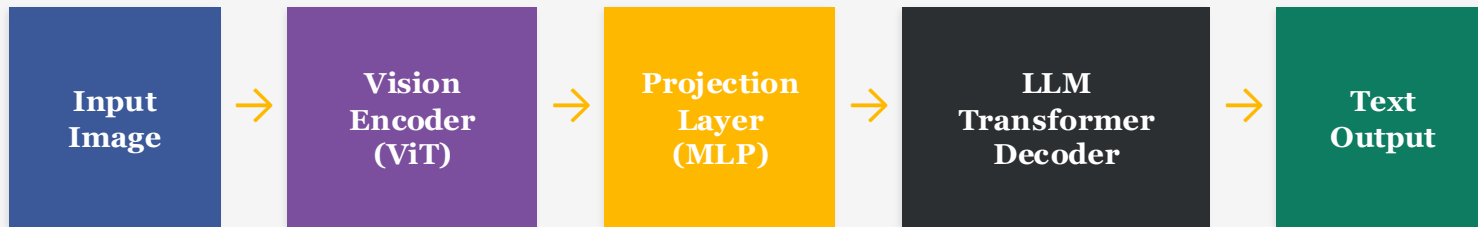
Sources: Ouyang et al. 2022 (InstructGPT); Bai et al. 2022 (Anthropic); DeepSeek-R1 2025

SECTION 2

Multi-Modality in LLMs

Teaching language models to see, hear, and understand

Adding Vision to a Language Model



Vision Encoder (ViT)

Splits image into 16×16 pixel patches. Each patch $\rightarrow$ embedding vector (like a 'visual word'). Pre-trained via CLIP or SigLIP on 400M+ image-text pairs.

Projection Layer

Maps vision embeddings into the LLM's token space. Often a simple 2-layer MLP. This is the 'translator' between visual and text languages.

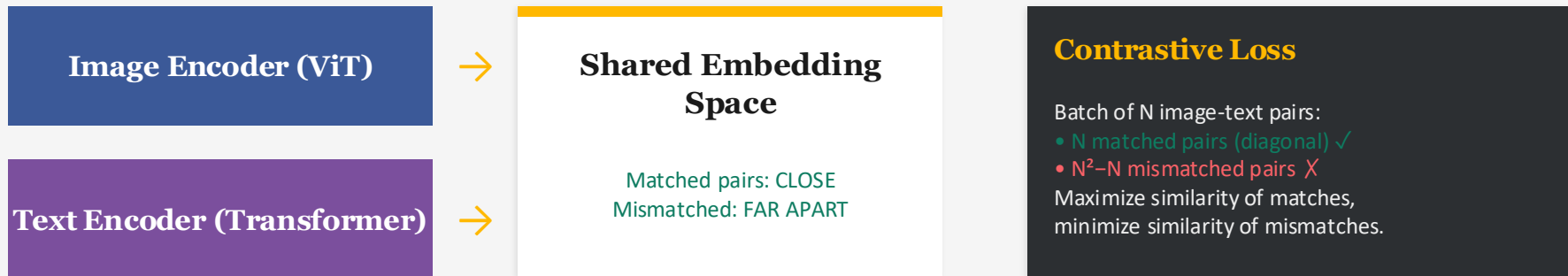
LLM Processes Both

The LLM receives interleaved text tokens and visual tokens. It reasons over both using the same attention mechanism. The LLM itself barely changes.

Analogy: Adding Eyes to a Brain That Could Only Read

Brain (LLM) = brilliant scholar who only reads text. Eyes (ViT) = captures visual info. Optic nerve (Projection) = translates visual signals into language. The brain's reasoning doesn't change — just gets new input.

CLIP: How Contrastive Pre-Training Works



Model	Organization	Modalities	Architecture Approach
GPT-4V / 4o	OpenAI	Text, Image, Audio, Video	Native multi-modal, dense visual tokens
Gemini 2.0	Google DeepMind	Text, Image, Audio, Video	Single unified token stream (all modalities)
Claude 3.5 / 4	Anthropic	Text, Image	Vision encoder + projection layer
LLaVA 1.6	UW-Madison	Text, Image	CLIP ViT + MLP projection + Vicuna
Llama 3.2 Vision	Meta AI	Text, Image	Cross-attention adapters (LLM frozen)

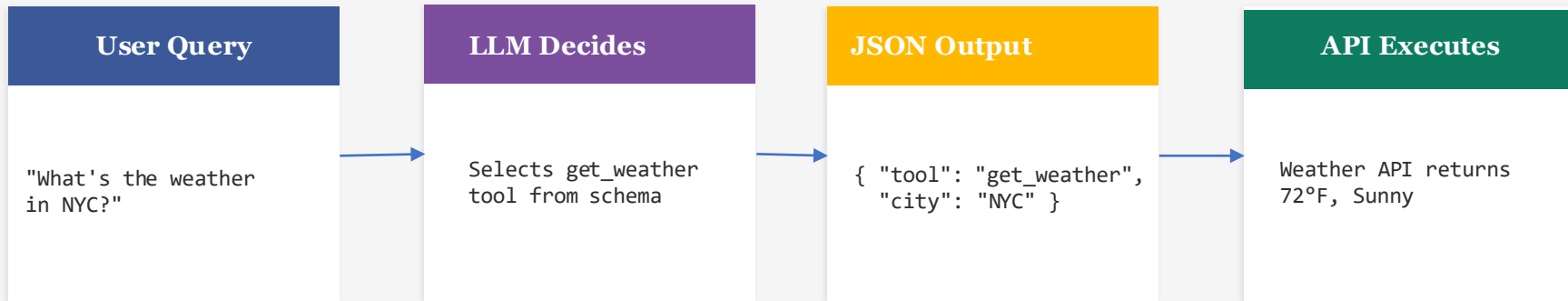
Sources: CLIP (Radford et al., 2021); SigLIP (Zhai et al., 2023); LLaVA (Liu et al., 2023); Gemini (Google, 2023); Llama 3.2 (Meta, 2024)

SECTION 3

RLHF in Practice

Function Calling — teaching models to use tools

Function Calling: How It Works



LLM Final Response:

"It's currently 72°F and sunny in New York City! Great weather for a walk."

Why Function Calling Matters

- Connects LLMs to real-time data & APIs
- Enables actions (booking, searching, computing)
- Reduces hallucination via grounded responses
- Powers AI agents and autonomous workflows

Model Context Protocol (MCP)

Anthropic's open standard (2024) for tool integration. Standardizes how LLMs discover and call tools. Adopted by OpenAI, Google. Pre-built servers for Slack, GitHub, databases.

Sources: OpenAI Function Calling (2023); Anthropic Tool Use (2024); MCP (Anthropic, 2024); Gorilla (Patil et al., 2023)

How RLHF Trains Function Calling

Stage 1: SFT on Tool Trajectories

Train on examples of correct tool use:

- When to call a tool vs. answer directly
- Which tool for which query
- Correct JSON formatting
- Proper argument selection

Stage 2: Reward Model Training

Humans rank tool-use trajectories:

- Correct tool > wrong tool
- Good arguments > bad arguments
- Using tools > hallucinating answers
- Reward model learns these preferences

Stage 3: PPO Optimization

Optimize against reward model:

- Maximize expected reward
- Stay close to SFT baseline
- Iterate: sample → score → update
- Model gets increasingly skilled

Key Research

Toolformer (Schick et al., 2023): Self-labeling — model teaches itself when tool calls reduce prediction loss. No expensive annotations needed.

Gorilla (Patil et al., 2023): Fine-tuned LLaMA for API selection → surpassed GPT-4 (94.2% vs 63.2%). Smaller specialized model beats larger general model.



Analogy: Teaching a Librarian to Use Reference Books

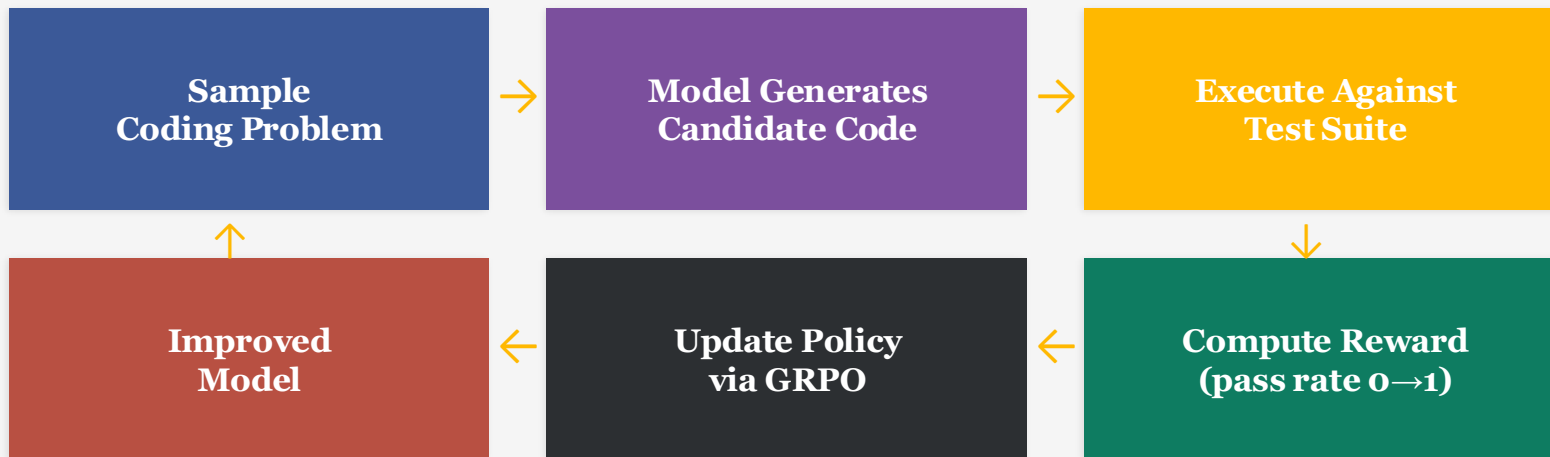
SFT = showing which books exist. Reward model = learning what makes a good lookup. PPO = practice until the librarian knows which source is best.

SECTION 4

RLVR in Practice

Code generation with verifiable rewards

Code Generation: The RLVR Training Loop



⌞⌟ Why Code is the Perfect RLVR Domain

Compilation check: does it parse? • Unit tests: do test cases pass? • Type checking: are types correct? • Performance: meets time/space constraints? • No human judgment needed — the computer decides.



Analogy: A Student Who Checks Their Own Homework

Write solution → check with answer key → learn from mistakes → try again. No teacher needed. Over millions of attempts: expert problem-solver.

Emergent Behaviors: Chain-of-Thought in Code

Before RLVR Training

Prompt: Sort a list in $O(n \log n)$

```
def sort(arr):  
    return sorted(arr) # just calls built-in
```

*No reasoning, no edge case handling,
no understanding of the algorithm.*

After RLVR Training

<think>

I need merge sort for $O(n \log n)$.

Base case: $\text{len} \leq 1 \rightarrow \text{return}$.

Split array, recurse, merge.

Edge cases: empty list, single element.

Let me verify with $[3,1,2] \rightarrow [1,2,3]$ ✓

</think>

Plans, reasons, verifies, handles edges

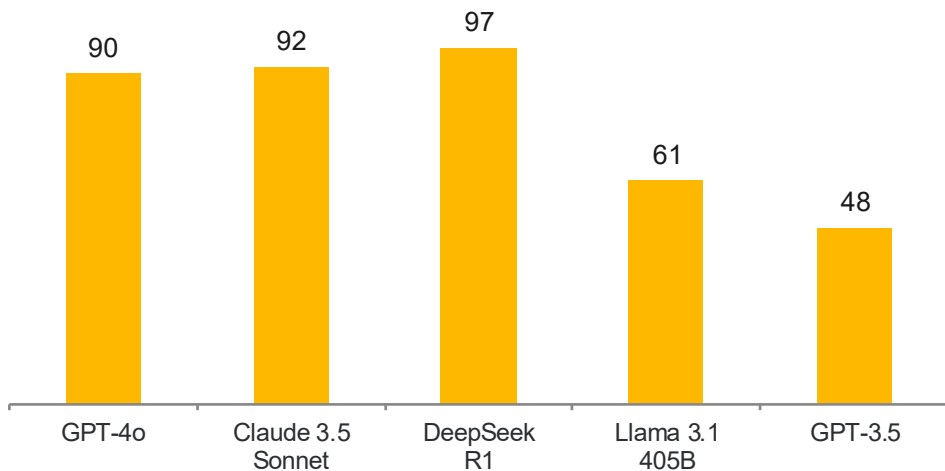


Why does this emerge?

The reward is: 'final answer must be correct.' The model discovers that thinking through steps $\rightarrow$ fewer errors $\rightarrow$ higher reward. So it spontaneously develops planning, verification, and backtracking. Nobody taught it to reason — it learned reasoning is useful.

Code Generation: Benchmarks & Results

HumanEval Pass@1 (%)



DeepSeek-R1 achieves 97.3% on HumanEval using pure RLVR — highest among all models tested.

Benchmark Evolution

HumanEval

164 function-level problems

Basic

MBPP

1,000 entry-level Python problems

Basic+

SWE-Bench

Real GitHub issues in full repos

Advanced

LiveCodeBench

Live competitive problems (anti-contamination)

Expert

SECTION 5

Capabilities Gained

What these training approaches unlock together

Capabilities Unlocked by Each Training Approach



Reasoning

Chain-of-thought, step-by-step problem solving, self-correction

RLVR



Tool Use

Function calling, API interaction, agent workflows

RLHF



Instruction Following

Precise adherence to complex, multi-step instructions

SFT



Visual Understanding

Document parsing, image analysis, chart reading

Multi-Modal



Code Generation

Writing, debugging, and explaining code with verification

RLVR + SFT



Safety & Alignment

Refusing harmful requests, honest uncertainty expression

RLHF / CAI

The Full Picture: The Capability Stack

Safety & Alignment Layer

RLHF → Helpful, Harmless, Honest | Constitutional AI → Principle-guided safety

Reasoning & Verification Layer

RLVR → Chain-of-thought | GRPO → Code correctness | Emergent self-correction

Instruction Following Layer

SFT → Task alignment | Format adherence | Multi-turn dialogue

Multi-Modal Layer

ViT encoder → Image patches | CLIP → Shared embeddings | Projection → LLM fusion

Foundation Layer

Pre-Training → Next-token prediction on 15T+ tokens | Language, facts, reasoning, code knowledge

Key Takeaways

- 1 Pre-training creates the knowledge foundation through next-token prediction on trillions of tokens
- 2 SFT teaches instruction-following by training on curated (instruction, response) pairs
- 3 RLHF aligns models with human values through reward modeling + PPO optimization
- 4 RLVR enables scalable reasoning training with verifiable rewards — no humans needed
- 5 Multi-modality adds vision by projecting image patches into the LLM's token space
- 6 Function calling (RLHF) and code generation (RLVR) are key practical applications
- 7 Chain-of-thought reasoning EMERGES naturally from RLVR training — nobody teaches it

References & Further Reading

1. Ouyang et al. (2022) "Training language models to follow instructions with human feedback" — InstructGPT
2. Bai et al. (2022) "Training a Helpful and Harmless Assistant with RLHF" — Anthropic
3. Bai et al. (2022) "Constitutional AI: Harmlessness from AI Feedback" — Anthropic
4. DeepSeek AI (2025) "DeepSeek-R1: Incentivizing Reasoning via RL" — Nature
5. Liu et al. (2023) "Visual Instruction Tuning" — LLaVA, NeurIPS Oral
6. Radford et al. (2021) "Learning Transferable Visual Models From Natural Language Supervision" — CLIP
7. Schick et al. (2023) "Toolformer: Language Models Can Teach Themselves to Use Tools"
8. Patil et al. (2023) "Gorilla: Large Language Model Connected with Massive APIs"
9. Chen et al. (2021) "Evaluating Large Language Models Trained on Code" — HumanEval
10. Schulman et al. (2017) "Proximal Policy Optimization Algorithms" — PPO
11. Wei et al. (2022) "Scaling Instruction-Finetuned Language Models" — FLAN
12. Jimenez et al. (2024) "SWE-bench: Can LMs Resolve Real-World GitHub Issues?"
13. Dosovitskiy et al. (2021) "An Image is Worth 16×16 Words" — Vision Transformer
14. Touvron et al. (2024) "The Llama 3 Herd of Models" — Meta AI

Thank You

Questions & Discussion

Next Class: Attention Mechanisms, Transformer Architecture & Scaling Laws

Instructor: Naresh Shah | nareshshah139@gmail.com

